

2025 JSSP-RUBI 프로젝트 온보딩 가이드

목차

1. 이 프로젝트는 무엇인가요?
 - 1.1 풀고자 하는 문제: Job Shop Scheduling Problem (JSSP)
 - 1.2 RUBI란?
 2. 스크립트 방식 vs 모듈화 방식 — 왜 바꿨을까요?
 - 2.1 “스크립트 방식”이란?
 - 2.2 스크립트 방식의 문제점
 - 2.3 모듈화 방식이 해결하는 것
 3. 프로젝트 구조 한눈에 보기
 4. 핵심 모듈 상세 설명
 - 4.1 simulator.py — “심판”
 - 4.2 dataset.py — “문제지 배포”
 - 4.3 individual.py — “해 하나”
 - 4.4 population.py — “집단 관리 + 초기화 팩토리”
 - 4.5 engine.py — “GA 엔진”
 - 4.6 crossover.py / mutation.py / selection.py — “GA 연산자”
 5. 실행 흐름 이해하기
 - 5.1 전체 흐름
 - 5.2 run_mode 4가지
 - 5.3 CLI 파라미터 덮어쓰기 (Override)
 6. 빠르게 실행해보기 (Quick Start)
 - 6.1 환경 설정
 - 6.2 첫 번째 실행
 - 6.3 결과 확인
 - 6.4 간트 차트 생성
 7. 설정 파일(config.json) 파라미터 레퍼런스
init_modes 숫자 코드 대응표
 8. 결과물 이해하기
 - 8.1 results.csv 컬럼 설명
 - 8.2 재현성 보장
 - 8.3 결과를 여러 번 실행해도 덮어쓰지 않는 이유
- 부록: 주요 용어 정리

2025 JSSP-RUBI 프로젝트 온보딩 가이드

작성: 백지원 | 날짜: 2026-05-04 | 소속: 서울대학교 조선해양공학과 생산공학연구실
스크립트 단위 코딩에 익숙한 연구원이 모듈화된 GA 연구 코드를 이해하고 실행할 수 있도록 안내하는 문서입니다.

목차

- 1. 이 프로젝트는 무엇인가요?
- 2. 스크립트 방식 vs 모듈화 방식 — 왜 바꿨을까요?
- 3. 프로젝트 구조 한눈에 보기
- 4. 핵심 모듈 상세 설명
- 5. 실행 흐름 이해하기
- 6. 빠르게 실행해보기 (Quick Start)
- 7. 설정 파일(config.json) 파라미터 레퍼런스
- 8. 결과물 이해하기
- 9. 부록: 주요 용어 정리

1. 이 프로젝트는 무엇인가요?

1.1 풀고자 하는 문제: Job Shop Scheduling Problem (JSSP)

JSSP는 n 개의 Job을 m 개의 Machine에서 처리하는 최적 순서를 찾는 문제입니다.
각 Job은 정해진 공정 순서(precedence constraint)에 따라 기계를 사용해야 하고, 각 기계는 한 번에 하나의 Job만 처리할 수 있어요. 목표는 모든 Job이 끝나는 시간(Makespan)을 최소화하는 것입니다.

용어	의미
Job	처리해야 할 작업 단위 (예: 제품 한 개)
Machine	작업을 수행하는 기계
Operation	Job의 각 단계 (Job ii의 kk번째 공정)
Makespan	모든 Job이 완료되는 최종 시각 (최소화 대상)
Sequence	각 기계에서 Job을 처리하는 순서

예를 들어, 3개의 Job과 3개의 Machine이 있을 때 어떤 순서로 작업을 배정하느냐에 따라 Makespan이 크게 달라집니다. 이 순서를 잘 결정하는 것이 핵심이에요.

JSSP는 NP-hard 문제로, 문제 크기가 커지면 완전 탐색이 불가능합니다. 이 프로젝트는 **유전 알고리즘(Genetic Algorithm, GA)**을 사용해 좋은 해를 빠르게 찾습니다.

1.2 RUBI란?

RUBI (Rule-Based Initialization)는 이 연구에서 개발한 초기 해(population) 생성 방법입니다. 완전히 무작위로 초기 해를 만드는 대신, 기계의 사용 순서와 작업 흐름을 고려해서 훨씬 좋은 출발점을 제공합니다.

GA의 최종 성능은 **초기 집단의 품질**에 크게 의존하기 때문에, 이 프로젝트의 핵심 연구 질문은 다음과 같아요:

“어떤 초기화 방법이 GA 최종 성능에 가장 긍정적인 영향을 줄까?”

이를 실험하기 위해 RANDOM, RUBI, MoRUBI, SPT, LPT, GT 등 여러 초기화 방법을 비교합니다.

2. 스크립트 방식 vs 모듈화 방식 — 왜 바꿨을까요?

이 섹션이 이 문서에서 가장 중요한 부분입니다. 코드를 처음 접하는 분들이 “왜 이렇게 파일이 많지?”라고 느끼는 이유를 설명합니다.

2.1 “스크립트 방식”이란?

처음 연구 코드를 작성할 때 대부분 이런 식으로 시작합니다:

```
# run_experiment.py (스크립트 방식 - 나쁜 예)
import random

n_job, n_machine, pop_size, generations = 10, 10, 100, 200

# 초기 집단 생성
population = [random.sample(range(n_job * n_machine), n_job * n_machine)
               for _ in range(pop_size)]

# 평가, 선택, 교배, 변이가 전부 한 파일에...
for gen in range(generations):
    # ... 300줄의 GA 코드 ...
    pass

# 결과 저장
with open("results.csv", "w") as f:
    f.write(f"{best_makespan}\n")
```

한 파일에 모든 것이 들어있어서 **처음에는 편하게 느껴집니다**. 하지만 연구가 진행될수록 문제가 생깁니다.

2.2 스크립트 방식의 문제점

문제	실제 상황	결과
재현 불가	“저번 실험 파라미터가 뭐였더라?”	결과는 있는데 설정을 기억 못함
실험 확장	초기화 방법 3가지를 비교하려면?	스크립트를 3번 복사해서 실행
코드 중복	비슷한 실험마다 파일을 조금씩 수정	어떤 파일이 최신 버전인지 모름
버그 전파	시뮬레이션 함수에 버그 발견	복사한 파일 전부 수정해야 함
협업 불가	다른 연구원에게 코드를 넘길 때	“어디서부터 읽어야 하나요?”

2.3 모듈화 방식이 해결하는 것

이 프로젝트는 **역할별로 파일을 분리**해서 각 모듈이 딱 하나의 책임만 지도록 설계했습니다.

실험 실행 흐름:

```
graph TD
    cli[cli.py] --> workflows[workflows.py]
    workflows --> engine[engine.py]
    engine --> crossover[crossover.py]
    engine --> mutation[mutation.py]
    engine --> selection[selection.py]
    crossover --> individual[individual.py]
    mutation --> individual
    selection --> individual
    individual --> simulator[simulator.py]
```

← 진입점: 인자 파싱, 설정 로드

← 조율: run_mode에 따라 분기

← GA 엔진: 진화 루프 실행

← GA 연산자 (교체 가능)

← 개체: 해 표현 + Makespan 계산

← 심판: 주어진 순서의 Makespan 계산

이 구조의 핵심 장점은 다음 세 가지입니다:

- 1. 재현성:** 실행할 때마다 설정이 `config.json` 으로 자동 저장됩니다. 6개월 후에도 그 파일 하나로 동일한 실험을 재실행할 수 있어요.
- 2. 독립성:** `selection.py` 를 바꿔도 `simulator.py` 는 전혀 영향을 받지 않습니다. 교배 연산자를 새로 추가할 때 다른 파일을 건드릴 필요가 없어요.
- 3. 단일 진입점:** `python -m src.runners.cli --config configs/default.json` 명령 하나로 모든 실험이 실행됩니다. 파일을 수정하거나 복사할 필요가 없어요.

3. 프로젝트 구조 한눈에 보기

```
2025_JSSP_RUBI/
|
+-- configs/
|   +-- default.json      ← 실험 설정 파일 (파라미터 조정은 여기서)
|
+-- data/
|   +-- JSPLIB/
|       +-- la01.txt      ← 벤치마크 데이터셋 (Lawrence 인스턴스)
|
+-- docs/                 ← 문서 및 렌더링 도구
|
+-- output/               ← 실험 결과 (자동 생성됨, 건드리지 않아도 됨)
|   +-- {run_name}/
|       +-- {timestamp}/
|           +-- config.json ← 실행 당시 설정 자동 저장
|           +-- results.csv ← 실험 결과 (행 단위로 추가됨)
|
+-- src/
|   +-- data/
|       +-- dataset.py    ← 데이터셋 파일 로드 및 파싱
|       +-- datagenerator.py ← 랜덤 인스턴스 생성 (선택적)
|
|   +-- environment/
|       +-- simulator.py  ← JSSP 시뮬레이터 (Makespan 계산 핵심)
|
|   +-- gas/              ← Genetic Algorithm 구성 요소
|       +-- individual.py ← 개체(Individual) 표현 및 평가
|       +-- population.py ← 집단 관리 + 초기화 팩토리
|       +-- engine.py     ← GA 메인 루프 (GAConfig + GAEngine)
|       +-- crossover.py  ← 교배 연산자
|       +-- mutation.py   ← 변이 연산자
|       +-- selection.py  ← 선택 연산자
|       +-- local_search.py ← 지역 탐색 (선택적)
|
|   +-- runners/
|       +-- cli.py        ← 명령줄 진입점
|       +-- workflows.py  ← run_mode별 실험 워크플로우
|
|   +-- visualization/
|       +-- gantt.py      ← 간트 차트 시각화
```

핵심 원칙: `configs/` 에서 설정하고, `src/runners/cli.py` 로 실행하면 됩니다. 나머지 파일들은 직접 건드릴 필요가 없어요.

4. 핵심 모듈 상세 설명

각 모듈이 어떤 역할을 하는지, 실제 코드와 함께 설명합니다.

4.1 simulator.py – “심판”

역할: 주어진 기계별 작업 순서(`machine_order`)로 실제 시뮬레이션을 실행해 Makespan을 계산합니다.

```
from src.environment.simulator import simulate

makespan, workingtime_log = simulate(
    op_data,          # 각 공정의 (기계번호, 처리시간) 정보
    machine_order,    # 각 기계에서의 Job 처리 순서 리스트
    n_job,
    n_machine,
)
# makespan: 완료 시각 (낮을수록 좋음)
# workingtime_log: 간트 차트용 로그 {machine: [(job, start, finish), ...]}
```

이 함수는 프로젝트의 “심판”입니다. GA가 만들어낸 해가 얼마나 좋은지를 판단하는 유일한 기준이에요. 모든 Individual은 결국 이 함수를 호출해서 자신의 점수를 받습니다.

4.2 dataset.py – “문제지 배포”

역할: 텍스트 파일로 된 JSSP 인스턴스를 읽어서 파이썬 객체로 변환합니다.

```
from src.data.dataset import Dataset

dataset = Dataset("data/JSPLIB/la01.txt")

print(dataset.n_job)      # 10 (Job 수)
print(dataset.n_machine) # 5  (Machine 수)
print(dataset.n_op)       # 50 (전체 Operation 수 = n_job * n_machine)

# op_data[job][op_idx] = (기계번호, 처리시간)
print(dataset.op_data[0][0]) # 예: (2, 21) - Job 0의 첫 공정: 기계 2에서 21분
```

데이터 파일 형식 (la01.txt 일부):

```
10 5                ← n_job  n_machine
0 21 1 53 4 95 3 55 2 34 ← Job 0: (기계 처리시간) 쌍 반복
3 77 1 55 0 42 4 13 2 93 ← Job 1
...
```

4.3 individual.py – “해 하나”

역할: GA에서 다루는 해(Solution) 하나를 표현합니다. 객체가 생성되는 순간 자동으로 Makespan을 계산해요.

```

from src.gas.individual import Individual

# seq: Operation 인덱스의 순열 (길이 = n_job * n_machine)
# 예) n_job=3, n_machine=3 이면 0~8 중 하나씩, 총 9개 원소
individual = Individual(config, seq=[2, 0, 5, 1, 4, 8, 3, 6, 7], op_data=op_data)

print(individual.makespan) # 예: 666 (낮을수록 좋음)
print(individual.fitness)  # 1/makespan 기반의 fitness 값

```

Individual 객체가 생성되면 내부적으로 4단계를 자동 실행합니다:

단계	변환	설명
1	seq → job_seq	Operation 인덱스를 Job 번호로 변환
2	job_seq → feasible_seq	공정 순서 제약(precedence) 적용
3	feasible_seq → machine_order	기계별 Job 처리 순서로 재배열
4	machine_order → simulate()	시뮬레이션 실행 → makespan 계산

4.4 population.py – “집단 관리 + 초기화 팩토리”

역할: **Individual** 여러 개로 구성된 **집단(Population)** 을 관리하고, 다양한 방법으로 초기 집단을 생성하는 팩토리 메서드를 제공합니다.

```

from src.gas.population import Population

# 방법 1: 완전 무작위 초기화
pop = Population(config, op_data, random_seed=42)

# 방법 2: RUBI 초기화 (Machine-I/O 기반 규칙 휴리스틱)
pop = Population.from_mio(config, op_data, "data/JSPLIB/la01.txt", random_seed=42)

# 방법 3: Giffler-Thompson 초기화 (능동 스케줄 생성)
pop = Population.from_GT(config, op_data, "data/JSPLIB/la01.txt")

```

초기화 방법	함수	설명	특징
<code>RANDOM</code>	<code>Population()</code>	완전 무작위 순열	빠르지만 품질 낮음
<code>RUBI</code>	<code>from_mio()</code>	Machine-IO 기반 규칙	기계 흐름 고려한 휴리스틱
<code>MoRUBI</code>	<code>from_modified_rubi()</code>	RUBI의 가중치 변형	처리시간 역수로 가중 샘플링
<code>SPT</code>	<code>from_SPT()</code>	Shortest Processing Time	짧은 작업 우선 배정
<code>LPT</code>	<code>from_LPT()</code>	Longest Processing Time	긴 작업 우선 배정
<code>GT</code>	<code>from_GT()</code>	Giffler-Thompson 알고리즘	고전적 능동 스케줄 생성법

4.5 engine.py – “GA 엔진”

역할: 선택 → 교배 → 변이 → 엘리트 보존의 **GA 루프**를 실행합니다.


```

from src.gas.engine import GAConfig, GAEngine
from src.gas.crossover import PMXCrossover
from src.gas.mutation import GeneralMutation
from src.gas.selection import RouletteSelection

# 설정 객체 생성
config = GAConfig(
    n_job=10, n_machine=5, n_op=50,
    population_size=100,
    generations=200,
    elite_ratio=0.05,          # 상위 5%를 다음 세대에 보존
    target_makespan=666,      # 이 값 이하 달성 시 조기 종료 (None이면 생략)
)

# 엔진 생성 + 초기화 동시 진행
engine = GAEngine(
    config=config,
    op_data=dataset.op_data,
    crossover=PMXCrossover(pc=0.9),
    mutation=GeneralMutation(pm=0.9),
    selection=RouletteSelection(),
    init_mode="RUBI",          # 초기화 방법 지정
    random_seed=42,
)

# 초기화 직후 품질 확인 (GA 실행 전)
print(engine.init_best_makespan) # 집단 내 최적 Makespan (초기)
print(engine.init_mean_makespan) # 집단 평균 Makespan (초기)

# GA 실행
best_individual, elapsed_time = engine.evolve()
print(f"최적 Makespan: {best_individual.makespan}")
print(f"소요 시간: {elapsed_time:.2f}초")

```

`GAEngine` 이 생성될 때 `init_best_makespan` 과 `init_mean_makespan` 을 기록하는 이유는, 초기화 방법의 품질과 GA의 개선 폭을 분리해서 분석하기 위해서입니다.

4.6 crossover.py / mutation.py / selection.py – “GA 연산자”

세 파일은 GA의 **진화 연산자**를 정의합니다. 모두 추상 기반 클래스(ABC)를 상속하는 구조라, 새로운 연산자를 추가할 때 기존 코드를 전혀 건드리지 않아도 됩니다.

```

# crossover.py
class Crossover(ABC):          # 인터페이스 정의
    def cross(self, p1, p2): ...

class PMXCrossover(Crossover): # PMX 교배 (현재 기본값, pc로 확률 제어)
    def __init__(self, pc: float): ...

# mutation.py
class GeneralMutation(Mutation): # 위치 무작위 교환 (pm: 각 위치의 변이 확률)
    def __init__(self, pm: float): ...

# selection.py
class RouletteSelection(Selection): # 룰렛 휠 선택 (fitness 비례 확률)
class TournamentSelection(Selection): # 토너먼트 선택 (k명 중 최적 선택)
    def __init__(self, tournament_size: int = 2): ...

```

새로운 교배 방법을 추가하고 싶다면 `crossover.py` 에 `Crossover` 를 상속하는 클래스만 추가하면 됩니다. 다른 파일은 건드리지 않아도 돼요.

5. 실행 흐름 이해하기

5.1 전체 흐름

```

python -m src.runners.cli --config configs/default.json
|
v
[cli.py: main()]
1. parse_args()          ← --config, --seed, --generations 등 파싱
2. load_config()          ← JSON 로드 + init_mode 정규화
3. _operator_classes()    ← 문자열 "PMX" → PMXCrossover 클래스 매핑
|
v
[workflows.py]           ← run_mode에 따라 분기
|-- "single_opt"          → run_single_opt()
|-- "single_comparison"   → run_single_comparison()
|-- "multiple_comparison" → run_multiple_comparison()
+-- "grid_search"         → run_grid_search()
|
v
[run_single_opt()]        ← 실험의 원자 단위(atomic unit)
1. Dataset 로드
2. GAConfig 생성
3. GAEngine 생성 (초기화 포함 -> init_best/mean 기록)
4. engine.evolve() 실행
5. results.csv에 결과 1행 추가(append)
6. config.json 저장 (최초 1회)

```

5.2 run_mode 4가지

run_mode	함수	언제 사용하나요?
single_opt	run_single_opt()	단일 초기화 방법 + 단일 seed로 딱 1회만 실행할 때
single_comparison	run_single_comparison()	여러 초기화 방법을 같은 seed로 비교할 때
multiple_comparison	run_multiple_comparison()	여러 데이터셋 x 여러 seed로 반복 실험할 때
grid_search	run_grid_search()	파라미터 조합 전체를 탐색(Grid Search)할 때

run_single_opt() 는 모든 run_mode의 원자 단위입니다. 나머지 세 함수는 이 함수를 조건에 따라 반복 호출하는 래퍼(wrapper)예요.

5.3 CLI 파라미터 덮어쓰기 (Override)

config.json 에 설정된 값을 실행할 때 명령줄에서 일부만 덮어쓸 수 있어요. 매번 파일을 편집하지 않아도 됩니다:

```
# generations만 500으로 바꿔서 실행
python -m src.runners.cli --config configs/default.json --generations 500

# 랜덤 시드 고정
python -m src.runners.cli --config configs/default.json --seed 123

# 결과를 다른 폴더에 저장
python -m src.runners.cli --config configs/default.json --output_root output/test_run
```

6. 빠르게 실행해보기 (Quick Start)

6.1 환경 설정

```
# 의존 패키지 설치
pip install numpy matplotlib tqdm
```

6.2 첫 번째 실행

configs/default.json 을 그대로 사용해서 바로 실행합니다:

```
# 프로젝트 루트(2025_JSSP_RUBI/)에서 실행
python -m src.runners.cli --config configs/default.json
```

실행하면 다음과 같이 출력됩니다:

```

=====
Saved: output/experiment/2026_05_04_12_00_00
Original Config: configs/default.json
Resolved Config(Run config):
Initialization Method: RANDOM, RUBI, GT, SPT
Dataset: la01.txt
=====

```

6.3 결과 확인

실행이 끝나면 **output/** 폴더에 결과가 자동으로 저장됩니다:

```

output/experiment/
+-- 2026_05_04_12_00_00/
    +-- config.json      ← 이 실행에 사용된 정확한 설정 (자동 저장)
    +-- results.csv      ← 실험 결과 표

```

results.csv 예시 (default.json 기준):

file	init_mode	seed	best_makespan_init	mean_makespan_init	best_makespan_final	elapsed_time
la01.txt	RANDOM	42	1024	1187.3	812	4.32
la01.txt	RUBI	42	887	953.1	791	4.51
la01.txt	GT	42	862	862.0	776	4.48
la01.txt	SPT	42	995	995.0	804	4.39

6.4 간트 차트 생성

configs/default.json 에서 **save_gantt** 를 **true** 로 변경하면 최적해의 간트 차트가 PNG 파일로 저장됩니다:

```

{
  "save_gantt": true
}

```

7. 설정 파일(config.json) 파라미터 레퍼런스

configs/default.json 전체 내용과 각 파라미터 설명입니다.

```

{
  "run_mode": "single_comparison",
  "run_name": "experiment",
  "datasets": ["la01.txt"],
  "seeds": [42],
  "init_modes": ["RANDOM", "RUBI", "GT", "SPT"],
  "population_size": 100,
  "generations": 200,
  "elite_ratio": 0.05,
  "crossover": {"type": "PMX", "pc": 0.9},
  "mutation": {"type": "General", "pm": 0.9},
  "selection": {"type": "Roulette"},
  "target_makespan": null,
  "save_gantt": false,
  "output_root": "output"
}

```

파라미터	타입	설명
<code>run_mode</code>	string	실험 종류: <code>single_opt</code> / <code>single_comparison</code> / <code>multiple_comparison</code> / <code>grid_search</code>
<code>run_name</code>	string	결과 저장 폴더명 (<code>output/{run_name}/</code> 아래에 저장)
<code>datasets</code>	list	사용할 데이터셋 파일명 (<code>data/JSPLIB/</code> 기준 또는 절대 경로)
<code>seeds</code>	list	랜덤 시드 목록 (재현성 확보용)
<code>init_modes</code>	list	비교할 초기화 방법 (숫자 1~6 또는 문자열 직접 입력 가능)
<code>population_size</code>	int	GA 집단 크기
<code>generations</code>	int	GA 최대 세대 수
<code>elite_ratio</code>	float	엘리트 보존 비율 (0.05 = 상위 5%를 다음 세대에 보존)
<code>crossover.pc</code>	float	교배 발생 확률 (0.0 ~ 1.0)
<code>mutation.pm</code>	float	각 위치의 변이 발생 확률 (0.0 ~ 1.0)
<code>target_makespan</code>	int or null	조기 종료 기준 (<code>null</code> 이면 세대 수 끝 까지 실행)
<code>save_gantt</code>	bool	최적해의 간트 차트 PNG 저장 여부
<code>output_root</code>	string	결과 저장 루트 디렉터리

init_modes 숫자 코드 대응표

`config.json` 에서 `init_modes`를 숫자로 써도 자동 변환됩니다:

숫자	문자열	알고리즘
1	<code>RANDOM</code>	완전 무작위 순열
2	<code>RUBI</code>	Machine-IO 기반 규칙
3	<code>MoRUBI</code>	처리시간 가중 RUBI
4	<code>SPT</code>	Shortest Processing Time
5	<code>LPT</code>	Longest Processing Time
6	<code>GT</code>	Giffler-Thompson 알고리즘

8. 결과물 이해하기

8.1 results.csv 컬럼 설명

컬럼	설명
<code>file</code>	사용한 데이터셋 파일명
<code>init_mode</code>	초기화 방법
<code>seed</code>	랜덤 시드 (같은 seed + 같은 설정 = 동일 결과 재현)
<code>best_makespan_init</code>	초기화 직후 집단 내 최적 Makespan
<code>mean_makespan_init</code>	초기화 직후 집단 평균 Makespan
<code>best_makespan_final</code>	GA 종료 후 최적 Makespan (낮을수록 좋음)
<code>elapsed_time</code>	실행 소요 시간 (초)

`best_makespan_init` 과 `best_makespan_final` 의 차이가 **GA의 개선 폭**입니다. `init_mode` 에 따라 출발점이 달라지므로 이 두 값을 함께 분석하는 것이 중요해요.

8.2 재현성 보장

6개월 후에도 동일한 실험을 재실행하려면 자동 저장된 `config.json` 을 사용하세요:

```
# 결과 폴더 안의 config.json으로 동일 실험 재실행
python -m src.runners.cli --config output/experiment/2026_05_04_12_00_00/config.json
```

이것이 모듈화 구조의 큰 장점입니다. 스크립트 방식에서는 “그때 파라미터가 뭐였더라?”를 기억해야 했지만, 이 프로젝트는 실행과 동시에 설정이 자동으로 기록됩니다.

8.3 결과를 여러 번 실행해도 덮어쓰지 않는 이유

실행할 때마다 `{timestamp}` 하위 폴더가 새로 생성됩니다. 즉, 같은 `run_name` 으로 여러 번 실행해도 이전 결과가 사라지지 않아요:

```
output/experiment/  
+-- 2026_05_04_12_00_00/    <- 첫 번째 실행  
+-- 2026_05_04_14_30_00/    <- 두 번째 실행 (첫 번째 결과 유지)  
+-- 2026_05_04_17_00_00/    <- 세 번째 실행
```

단, 같은 run 내에서 여러 `init_mode` 나 `seed` 의 결과는 동일한 `results.csv` 에 행 단위로 누적됩니다.

부록: 주요 용어 정리

용어	설명
JSSP	Job Shop Scheduling Problem — 기계 배정 순서 최적화 문제
Makespan	모든 Job이 완료되는 시각 (최소화 목표)
GA	Genetic Algorithm (유전 알고리즘) — 진화를 모방한 최적화 방법
Individual	GA에서 해 하나를 나타내는 객체
Population	Individual 여러 개로 구성된 집단
Fitness	개체의 적합도 (이 프로젝트에서는 1/makespan 기반)
Selection	다음 세대 부모를 선택하는 연산
Crossover	두 부모에서 자식 해를 만드는 연산
Mutation	해의 일부를 무작위로 변경하는 연산
Elite	상위 개체 (elite_ratio 비율만큼 다음 세대에 그대로 전달)
RUBI	Rule-Based Initialization — 이 연구의 핵심 초기화 방법
MoRUBI	Modified RUBI — 처리시간 역수 기반 가중 샘플링 적용
SPT	Shortest Processing Time — 짧은 작업 우선 스케줄링
LPT	Longest Processing Time — 긴 작업 우선 스케줄링
GT	Giffler-Thompson 알고리즘 — 능동 스케줄(Active Schedule) 생성
PMX	Partially Mapped Crossover — 순열에 적합한 교배 방법
run_mode	실험 종류 (single_opt / single_comparison / multiple_comparison / grid_search)
seed	랜덤 시드 — 같은 seed + 같은 설정이면 결과가 동일하게 재현됨
elite_ratio	엘리트 보존 비율
Gantt Chart	기계별 작업 타임라인을 시각화한 차트
Precedence Constraint	각 Job의 공정 순서 제약 (앞 공정이 끝나야 다음 공정 가능)

이 문서에 대한 질문이나 오류는 baekjiwon@snu.ac.kr 로 보내주세요.